

E-BOOK TÉCNICO AVANÇADO

Engenharia de Harness

O que é, o que não é,
e por que a confusão custa caro

Por que comparar ferramentas não é engenharia,
e onde mora a variável que decide se o agente entrega.

SCORAS ACADEMY

A primeira formação continuada em Engenharia de IA do Brasil

Material didático. Fontes oficiais citadas ao final.



Capítulo 1 · Um erro de categoria

Existe uma crítica recorrente à engenharia de harness. Ela diz que o termo é taxonomia inventada, que por trás dele só existe prática antiga com nome novo, e que basta uma tarde lendo documentação para cobrir o assunto inteiro.

A crítica costuma vir com dados. Alguém roda o mesmo modelo dentro de CLIs diferentes, observa que os resultados ficam parecidos, e conclui que o harness é ruído.

Esse argumento tem uma origem identificável, e ela não é má-fé. Ele aparece com mais frequência entre profissionais de engenharia de software sólidos que ainda não conduziram projetos de engenharia de IA em produção. O mecanismo do erro é conhecido: diante de algo novo, a mente mapeia para o conceito familiar mais próximo. E o mapeamento acerta quase tudo, errando exatamente a parte que é nova.

As três reduções

REDUÇÃO 1

"É só um laço while." Verdade parcial. O laço existe há décadas. O que mudou é que o ator dentro dele não é determinístico, e por isso o laço deixou de ser controle de fluxo e virou sistema de controle.

REDUÇÃO 2

"É só teste e revisão." Verdade parcial. Verificação tem cinquenta anos. Só que aqui o resultado do teste não vai para um humano ler: ele volta para dentro da janela de contexto e determina a próxima ação da máquina.

REDUÇÃO 3

"É só escolher uma biblioteca." Verdade parcial. Escolher framework é uma decisão de dez minutos. Projetar o que o framework expõe ao modelo é o trabalho que sobra depois dela.

Cada redução preserva o vocabulário da engenharia de software determinística e descarta a única propriedade que muda o problema. Este e-book existe para nomear essa propriedade com precisão, mostrar onde ela aparece, e separar de forma operacional o que é engenharia de harness do que apenas se parece com ela.

Por que a confusão custa caro

Se fosse apenas disputa de vocabulário, não valeria um e-book. O problema é que a redução tem consequência operacional direta.

Um time que acredita que harness é escolha de ferramenta tenta resolver problema de agente trocando de CLI. Quando a troca não resolve, e ela não resolve, a conclusão seguinte costuma ser que o modelo ainda não está bom o bastante. O time então passa a esperar a próxima versão do modelo, em vez de mexer na única superfície que estava sob o controle dele.

É o mesmo padrão de quem culpa o banco de dados por uma lentidão que vinha da consulta. A causa fica no lugar onde ninguém olhou, porque o vocabulário disponível não tinha nome para ela.

O que você vai levar daqui

Uma definição verificável de harness, os cinco eixos de projeto, seis atividades que são confundidas com engenharia de harness, o desenho de experimento que separa opinião de medição, e um teste de uma pergunta que revela em qual dos dois lados você está.

Capítulo 2 · A definição

A definição mais útil é a mais direta, e vem de Vivek Trivedy, da LangChain: **agente = modelo + harness**. Se você não é o modelo, você é o harness.

O harness é todo pedaço de código, configuração e lógica de execução que não é o modelo em si. Um modelo cru não é um agente. Ele vira agente quando recebe estado, execução de ferramentas, laços de realimentação e limites aplicáveis.

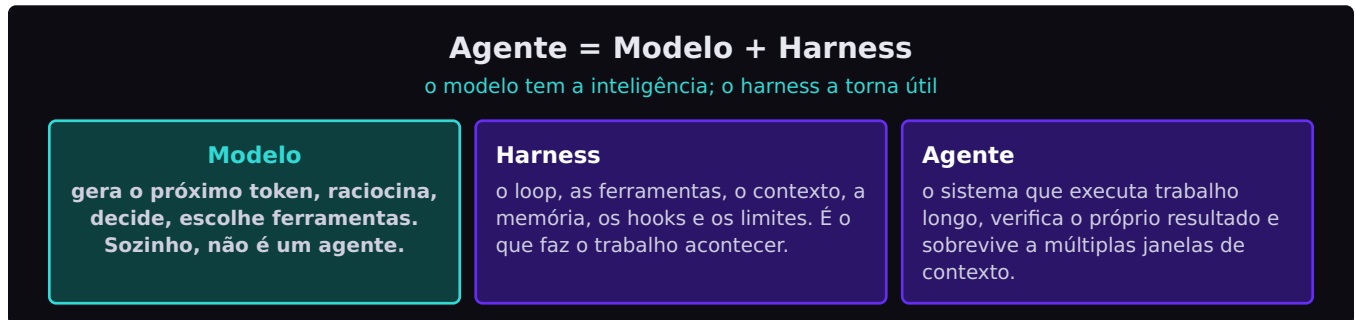


Figura 1. A equação fundamental. Diagrama Scoras Academy, baseado em langchain.com/blog.

2.1 Os cinco eixos de projeto

Dizer que o harness é "tudo que não é o modelo" é verdadeiro e pouco operacional. O que torna a definição utilizável é decompô-la em eixos que podem ser projetados e medidos um a um.

Eixo	Decisões que ele contém
System prompt	papel, limites, política de parada, como usar os outros quatro eixos
Contrato de ferramentas	tools, skills e MCPs, suas descrições, o esquema de entrada, o retorno em caso de sucesso e o retorno em caso de falha
Infraestrutura acoplada	filesystem, sandbox, browser, runtimes e pacotes pré-instalados
Lógica de orquestração	spawn de sub-agentes, handoffs, roteamento de modelo, isolamento de contexto
Hooks e middleware	compactação, continuação, gate de verificação na saída, offloading de resultado grande

Guarde esta tabela. Ela é o critério que usaremos no capítulo 4 para julgar se uma atividade é engenharia de harness ou outra coisa. A pergunta será sempre a mesma: **qual desses eixos foi manipulado?**

2.2 Harness e framework não são a mesma coisa

Framework é a ferramenta com que você constrói. Harness é a arquitetura resultante ao redor do modelo. Deep Agents é um framework que já entrega um harness pronto; você também pode construir o seu do zero sobre um runtime de grafo. Confundir os dois é o que faz alguém achar que a decisão acabou quando o pacote foi instalado.

Engenharia de contexto não é sinônimo

Engenharia de contexto cuida do que o modelo vê a cada passo. É um subsistema dentro do harness, não o conjunto. O harness inclui ainda orquestração de ferramentas, persistência de estado, recuperação de erro, laços de verificação, segurança e ciclo de vida.

Capítulo 3 · A inversão que explica tudo

Existe uma única propriedade que separa engenharia de harness de engenharia de software convencional. Ela é simples de enunciar e muda todas as prioridades de projeto.

A inversão

No software determinístico, uma interface é um contrato para o programador. No sistema agêntico, a interface é contexto para o modelo.

Considere a mensagem de erro de uma função. Num programa comum, ela é texto morto: vai para um log, e um humano lê depois, se ler. Otimizar a redação de uma mensagem de erro é higiene, não arquitetura.

Dentro de um agente, essa mesma mensagem é um bloco de tokens injetado na janela de contexto, no instante da falha, imediatamente antes da decisão seguinte. Ela não descreve o que aconteceu: ela **determina o que vai acontecer**.

Artefato	Software determinístico	Sistema agêntico
Descrição da função	documentação, opcional	é o roteador: decide se a tool certa é chamada
Mensagem de erro	log para diagnóstico humano	entrada que determina a próxima ação
Formato do retorno	convenção de serialização	orçamento de contexto consumido
Ordem das operações	escrita pelo programador	escolhida pelo modelo, guiada pelo harness
Limite de permissão	checagem no código	ou é executável no servidor, ou é sugestão

Quem vem da engenharia de software determinística olha para a coluna do meio e conclui, com razão dentro daquele mundo, que nada ali é digno de uma disciplina. A coluna da direita é o objeto da engenharia de harness, e ela só existe porque o consumidor da interface passou a ser um ator não determinístico que lê texto.

3.1 A consequência prática

Se a interface é contexto, então cada decisão de interface é uma decisão de engenharia de contexto, e portanto uma decisão de desempenho. Isso reordena o que merece atenção.

PRIORIDADE ANTIGA

A tool funciona? Tem teste? Tem cobertura? Trata exceção sem derrubar o processo?

PRIORIDADE NOVA

A tool ensina o modelo a se recuperar? A falha nomeia a causa? O sucesso cabe no orçamento de contexto?

Repare que a lista da direita não substitui a da esquerda, e que nenhum item dela aparece numa revisão de código convencional, porque nenhum deles é sobre correção. São todos sobre o efeito que a interface produz num leitor não determinístico.

As duas listas importam. A primeira é pré-requisito e continua valendo integralmente. A segunda é o que a primeira não cobre, e é onde vive a variância de desempenho que a crítica não consegue explicar.

Capítulo 4 · O que não é engenharia de harness

Seis atividades legítimas que são confundidas com a disciplina. Nenhuma delas é inútil. Todas elas são outra coisa. O critério de julgamento é o do capítulo 2: qual eixo foi manipulado?

4.1 Comparar CLIs prontos

NÃO É ENGENHARIA DE HARNESS

Instalar Claude Code, Codex, Cursor e OpenCode, rodar a mesma tarefa nos quatro e comparar a pontuação.

Isso é benchmark de fornecedor, e é uma atividade útil. Responde a pergunta "qual CLI eu instalo". Não responde "como eu projeto o sistema", porque nenhum dos cinco eixos foi manipulado. Os quatro produtos diferem em centenas de decisões simultâneas, e o resultado agregado não atribui variância a nenhuma delas.

O equívoco tem uma forma mais sutil, e é aí que ele derruba conclusões. Quando o experimento roda em benchmark saturado, no qual modelos de fronteira já pontuam acima de 90, os harnesses empatam. A leitura ingênua é que o harness é irrelevante. A leitura correta é que **não existe margem para medir**. Sem headroom não há sinal, e ausência de sinal não é evidência de ausência de efeito.

A VERSÃO QUE VIRA ENGENHARIA

Fixe o modelo por proxy, fixe as tarefas, selecione apenas as tarefas com taxa de sucesso entre 20% e 80%, e então varie um eixo por vez dentro de um único harness. Aí você atribui a variância a uma decisão de projeto.

4.2 Testar se as ferramentas funcionam

NÃO É ENGENHARIA DE HARNESS

Escrever testes determinísticos que verificam se a função `buscar_web()` retorna o resultado esperado.

Isso é teste de unidade de código comum, e deve ser feito. Aliás, é a parte mais barata e mais negligenciada de um sistema de agentes. Mas testar se a ferramenta funciona é validar o artefato, não projetar o contrato.

A distinção é operacional. O teste de unidade responde "a função retorna o valor certo". A engenharia de harness pergunta: quando ela falha, **o que exatamente entra na janela de contexto?** Uma tool com cem por cento de cobertura de testes e mensagem de erro genérica é uma tool correta dentro de um harness ruim.

O mesmo vale para o outro lado do contrato, que é a descrição. Ela não é documentação: é o roteador que decide se o modelo chama a ferramenta certa. Compare as duas versões abaixo, funcionalmente idênticas.

```
# descricao vaga: passa em qualquer teste de unidade
"""Busca informacao."""

# descricao como roteador: diz quando usar, quando nao usar, e o que volta
"""Busca na documentacao interna do projeto (nao na web).
Use quando precisar de API, contrato ou decisao ja registrada.
Nao use para conhecimento geral. Retorna ate 5 trechos com o caminho
do arquivo de origem. Retorna lista vazia se nada casar."""
```

A segunda versão custa dois minutos e não muda uma linha do código executado. Ela muda a taxa de roteamento correto, que é métrica de desempenho do agente. No capítulo 5 está o número público que a Anthropic divulgou sobre exatamente esse eixo.

4.3 Instalar um framework de agentes

NÃO É ENGENHARIA DE HARNESS

`pip install deepagents`. Ou LangGraph, ADK, CrewAI, o que for.

Isso é adoção de harness pronto, e costuma ser a decisão certa. Usar um ORM não é modelagem de dados; usar um framework web não é arquitetura de sistemas. O framework entrega os quatro pilares embutidos, e a engenharia começa exatamente onde ele para: nas suas ferramentas, no seu contrato, nos seus limites, no seu gate de verificação.

4.4 Escrever um prompt melhor

NÃO É ENGENHARIA DE HARNESS

Reescrever o system prompt até o comportamento melhorar.

Isso é engenharia de prompt, que é parte da engenharia de contexto, que é um subsistema do harness. É um eixo de cinco, e frequentemente não é o de maior efeito. Um sinal claro de que o problema saiu do domínio do prompt: se o system prompt cresce a cada bug corrigido, você está usando instrução para resolver um problema de arquitetura.

4.5 Rodar o agente em laço até terminar

NÃO É ENGENHARIA DE HARNESS

Colocar o agente num `while` com uma suíte de testes no fim.

Aqui a crítica está mais próxima de acertar, e vale conceder o ponto. Executar, verificar e iterar é prática antiga com nome novo. O que não é antigo é o mecanismo de controle sobre um ator que pode declarar conclusão sem ter concluído.

O padrão que ficou conhecido como Ralph Loop, descrito publicamente por Geoffrey Huntley em julho de 2025, ilustra a diferença: um hook intercepta a tentativa de saída do modelo e reinjeta o objetivo original numa **janela de contexto limpa**, forçando a continuidade contra a meta. Isso não sai de "rode testes". Sai de projetar o loop como sistema de controle, com gate de saída, orçamento de turnos e estado persistido fora da janela.

4.6 Adicionar mais agentes

NÃO É ENGENHARIA DE HARNESS

Transformar um agente em cinco, com um orquestrador no topo.

Orquestração é um eixo de cinco, e frequentemente o eixo errado. A própria Anthropic registra que sistemas multi-agente consomem cerca de 15 vezes mais tokens que chat, e que domínios com contexto compartilhado ou muitas dependências entre agentes não são bons candidatos hoje. Código está nessa categoria.

Regra de leitura dos seis casos

Se você não consegue apontar qual dos cinco eixos mudou, e mostrar o que ficou constante enquanto ele mudava, você não fez engenharia de harness. Fez uso, adoção ou comparação de harness. Todas legítimas, nenhuma equivalente.

Capítulo 5 · A evidência pública

A afirmação de que o harness é ruído é empírica, e portanto falsificável. Existem resultados públicos que a contradizem, com modelo mantido constante.

Top 30 → 5

LangChain no Terminal-Bench 2.0, mesmo modelo, mesmo loop, mudando só o harness

40%

queda no tempo de conclusão após reescrever a descrição de uma tool, na Anthropic

85%

dos times em produção constroem harness próprio em vez de usar framework

5.1 O contraexemplo direto

O Terminal-Bench 2.0 é um leaderboard aberto, com 89 tarefas verificadas por testes determinísticos. A LangChain documentou a saída de fora do Top 30 para o Top 5 alterando apenas o harness, com modelo e loop ReAct constantes. Se o harness fosse ruído, não haveria o que mover 25 posições.

5.2 O eixo isolado, já medido

Um resultado ainda mais direto vem do relato de engenharia da Anthropic sobre seu sistema de pesquisa multi-agente. Eles descrevem um agente que pegava uma ferramenta MCP defeituosa e reescrevia a descrição dela. O efeito: queda de 40% no tempo de conclusão de tarefa para os agentes seguintes. A conclusão que eles registram é que descrições ruins mandam agentes por caminhos completamente errados.

Um eixo. Sem tocar no modelo. Quarenta por cento.

5.3 O que roda em produção

O estudo *Measuring Agents in Production*, de Pan e colegas, com Berkeley, IBM Research e Stanford, ouviu 306 praticantes e fez 20 estudos de caso em 26 domínios. Dois achados importam aqui.

O primeiro: 68% dos agentes em produção executam no máximo dez passos antes de intervenção humana. Isso costuma ser lido como prova de que loop simples basta. É o contrário: é o diagnóstico de um teto, e o teto tem causas conhecidas, que são parada precoce, context rot e ausência de verificação. Todas de nível de harness.

O segundo: a maioria dos times constrói implementação própria em vez de adotar framework, e o estudo conclui que confiabilidade é o principal desafio, endereçado com **design de nível de sistema**. Traduzindo: o mercado já faz engenharia de harness na mão. Só não chama assim.

5.4 O que essa evidência não prova

Honestidade metodológica é parte do argumento, então vale delimitar. Nada disso prova que qualquer mudança de harness melhora qualquer agente. Nem que harness importa igualmente em toda faixa de capacidade de modelo. Nem que multi-agente compensa.

O que os resultados sustentam é mais estreito e suficiente: com o modelo constante, decisões de harness produzem variação de desempenho grande o bastante para dominar o resultado. Isso basta para falsificar a afirmação de que harness é ruído, que é a afirmação em disputa.

O ponto que fecha o argumento

Quando o modelo vira commodity, e os modelos de fronteira já estão a poucos pontos uns dos outros, a variância de resultado migra para o que está ao redor deles. A commoditização do modelo é o argumento mais forte a favor da engenharia de harness, não contra.

Capítulo 6 · Como se mede um eixo

A diferença entre opinião e engenharia é o desenho experimental. Este é o protocolo mínimo para atribuir variância a uma decisão de projeto, e ele cabe em uma tarde.

1 Fixe tudo, menos um eixo

Modelo com versão travada, temperatura, conjunto de tarefas, loop, teto de turnos e timeout. Todo harness fala com o mesmo proxy, para que o modelo seja realmente a constante e não apenas nominalmente.

2 Selecione tarefas com margem

Rode um piloto e fique apenas com as tarefas cuja taxa de sucesso caia entre 20% e 80%. Fora dessa banda existe efeito teto ou efeito piso, e nenhum dos dois produz sinal. Congele a lista antes de começar.

3 Meça sucesso e turnos

Sucesso verificado por suíte de testes, nunca por LLM avaliador. Turnos até a conclusão é a métrica esquecida, e captura ganho de eficiência que o pass/fail esconde.

4 Intercale as condições

Desenho pareado, cada tarefa nas duas condições, ordem embaralhada com semente fixa. Rodar todas de A e depois todas de B cria efeito de ordem por cache de prompt, confundível com efeito de tratamento.

5 Reporte tamanho de efeito, não p-valor

Bootstrap pareado agrupado por tarefa para taxa de sucesso, com intervalo de confiança. Wilcoxon pareado sobre a mediana por tarefa para turnos, porque a distribuição é assimétrica e censurada no teto.

6 Teste o mecanismo, não só o efeito

Se a hipótese diz que o efeito vem de um mecanismo específico, o efeito precisa sumir onde o mecanismo não age. Se ele aparece nos dois grupos, existe contaminação no desenho e o resultado não é causal.

6.1 O passo que quase todo mundo pula

O sexto. É tentador parar no quinto, porque no quinto já existe um número publicável. Mas um efeito sem mecanismo testado é indistinguível de um artefato do desenho, e artefato de desenho é o modo mais comum de um experimento de agente produzir número bonito e falso.

7 Publique o resultado nulo também

Um experimento que só é publicado quando confirma a hipótese não é evidência, é seleção. Declare a hipótese, o n e o critério de falsificação antes de rodar, e publique o que sair.

6.2 O gate que salva o orçamento

Um detalhe prático que separa quem já rodou isso de quem apenas desenhou: coloque o teto de gasto num mecanismo que bloqueia, não num aviso. Chave com orçamento máximo no proxy, teto de turnos por execução e timeout. Um agente em laço consome o orçamento inteiro antes de alguém perceber, e o custo do susto costuma virar o motivo de o experimento nunca ser repetido.

Poder estatístico, dito na cara

Com 20 tarefas e 5 repetições, o desenho detecta diferenças na ordem de 15 pontos percentuais. Diferenças menores não são detectáveis nessa escala, e afirmar o contrário é erro. Declarar o próprio poder é parte do resultado, não rodapé.

Capítulo 7 · O eixo mais subestimado

Se você fosse escolher um único eixo para manipular primeiro, a aposta com melhor fundamento teórico não é o system prompt. É o **contrato de recusa** da ferramenta: o que ela devolve quando falha.

```
# Condição de controle: a tool falha e devolve isto
Error: command failed

# Condição de tratamento: mesma falha, contrato tipado
{
  "status": "refused",
  "code": "MISSING_DEPENDENCY",
  "detail": "python module 'pytest' is not installed",
  "next_action": "run: pip install pytest"
}
```

A diferença parece cosmética e não é. Um prompt é um prior: ele decai ao longo da janela e compete com tudo que vem depois. Uma recusa tipada é evidência entregue no ponto de decisão, no instante da falha, com saliência máxima.

Erro genérico obriga o modelo a adivinhar o que deu errado. Adivinhação é onde nasce a recuperação alucinada, que é o modo de falha mais caro de um agente autônomo, porque ele consome turnos e produz estado inconsistente antes de ser detectado.

7.1 O enquadramento formal

A Lei da Variedade Requisita, formulada por W. Ross Ashby em 1956, diz que um controlador precisa de variedade compatível com a variedade do sistema que controla. Só a variedade absorve variedade.

Aplicada aqui: o canal de erro é por onde a variedade do ambiente entra no agente. Uma única mensagem genérica para todos os modos de falha é um canal de variedade um. Ele limita o controlador por construção, independentemente de quão capaz seja o modelo.

A tradução de engenharia

Uma tool que devolve recusa tipada com causa nomeada e próxima ação converte recuperação alucinada em recuperação determinada. O modelo deixa de inferir e passa a executar.

Repare no que muda na prática. Com erro genérico, a próxima ação do modelo é uma hipótese sobre a causa. Com recusa tipada, a próxima ação já veio nomeada no payload, e o modelo executa em vez de inferir. O número de turnos até a conclusão cai antes mesmo de a taxa de sucesso subir, e é por isso que turnos precisa ser métrica primária.

7.2 A regra do classificador

Um detalhe metodológico que decide se o eixo é realmente único: a classificação da falha precisa ser determinística, feita por regra sobre stderr e código de saída. Nunca por um LLM. Se um modelo gera a recusa, o experimento passa a medir também a qualidade do classificador, e o eixo deixa de ser um.

O crédito e o cuidado

A formulação de recusas tipadas como eixo de maior impacto foi levantada publicamente por Laurent Poupet. O efeito provavelmente é condicional ao horizonte: em tarefa curta o agente quase não encontra recusa. Isso é hipótese, e hipótese se mede.

Capítulo 8 · O vocabulário técnico

Termos que aparecem no curso e que precisam significar a mesma coisa para todo mundo.

8.1 Context rot

Degradação de desempenho conforme a janela de contexto enche. Não é falta de espaço apenas: é ruído acumulado competindo com o sinal. É a causa mecânica de o agente repetir trabalho já feito e perder o objetivo original.

8.2 Os quatro movimentos da engenharia de contexto

Escrever. Tirar da janela. O agente grava resultado intermediário em arquivo e mantém no contexto apenas a referência.

Comprimir. Resumir o histórico antigo preservando decisões e descartando o processo que levou a elas.

Isolar. Dar janela própria a subtarefas pesadas, trazendo ao orquestrador só a conclusão limpa.

Selecionar. Trazer de volta só o trecho que a decisão atual exige, em vez de reinjetar tudo.

8.3 As três memórias

Memória	Onde vive	Responde a pergunta
De trabalho	na janela, via lista de todos	o que eu estava fazendo mesmo?
De sessão	no checkpointer do runtime	de onde eu retomo depois da interrupção?
De longo prazo	em arquivos, carregada sob demanda	o que eu já aprendi sobre este projeto?

Misturar as três no mesmo lugar é o caminho mais rápido para um agente confuso e caro.

8.4 A progressão da disciplina

Engenharia de prompt cuida da instrução. Engenharia de loop cuida do ciclo de execução e da condição de parada. Engenharia de harness contém as duas e acrescenta o que nenhuma delas alcança: contrato de ferramenta, infraestrutura, orquestração e hooks determinísticos.

A progressão não é substituição. É contenção. Cada camada continua valendo dentro da seguinte, e a confusão que este e-book combate nasce justamente de tomar a parte pelo todo.

8.5 Restrição consultiva e restrição executável

Uma proibição escrita no system prompt é consultiva: depende de o modelo cooperar. Uma proibição no servidor que atende a chamada é executável: independe do modelo. As duas coexistem em sistemas sérios, e o trabalho de projeto é decidir qual restrição vive onde. Guardrail que só existe em prompt é a forma mais comum de falsa sensação de segurança em agentes.

8.5 Avaliação de trajetória

Avaliar agente não é avaliar modelo. O modelo produz um texto; o agente executa um percurso. Um agente pode chegar à resposta certa por sorte, depois de dez chamadas desnecessárias. Isso é falha de engenharia mesmo com resultado certo, porque custa caro e não se sustenta em escala. Meça sucesso da tarefa, custo por tarefa concluída e taxa de intervenção humana. Os três juntos contam a história real.

Capítulo 9 · O teste de uma pergunta

Existe um teste único que separa usar harness de construir harness, e ele cabe numa frase.

Onde estão escritas as proibições do seu agente?

Formulação de Laurent Poupet

Se a resposta é "no system prompt", você está usando um harness. Se a resposta é "no servidor que recusa a chamada", você está construindo um.

A distinção real é entre restrição consultiva e restrição executável. Um limite no prompt é uma esperança de que o modelo se comporte. Um limite na ferramenta é determinístico e independe do modelo. Sistemas sérios usam as duas em camadas, e o trabalho de projeto é decidir qual restrição vive onde.

Anti-padrões que denunciam o problema

1 O prompt que cresce a cada bug

Cada exceção nova vira mais uma linha de instrução. Se o system prompt virou uma lista de exceções, o problema é de arquitetura e está sendo tratado como instrução.

2 A verificação que não verifica

Pedir ao próprio modelo que julgue o resultado que ele acabou de produzir. O mesmo raciocínio que errou é usado para avaliar. Verificação útil é externa e determinística.

3 O loop sem freio

Iterar até resolver, sem teto de passos nem critério de parada verificável. Basta uma tarefa ambígua para o agente girar queimando tokens.

4 A memória que só cresce

Guardar tudo na esperança de que sirva depois. Resultado: recuperação lenta e contexto poluído. Memória de longo prazo exige curadoria, com regra de expiração.

5 A orquestração decorativa

Cinco agentes onde um resolveria, porque o diagrama fica bonito. Cada agente a mais adiciona arestas, contratos de mensagem e versões conflitantes da verdade.

Os cinco anti-padrões acima têm um traço em comum: em todos, uma decisão de arquitetura foi delegada ao modelo. O prompt inchado delega a política de exceção. A auto-verificação delega o julgamento. O loop sem freio delega a condição de parada. A memória sem curadoria delega a relevância. A orquestração decorativa delega a coordenação. Engenharia de harness é, em boa medida, decidir o que não delegar.

Checklist de um harness saudável

- | | |
|---|---|
| ☐ Existe plano persistido que mantém o objetivo vivo ao longo de muitos passos | ☐ Existe gate de verificação determinístico antes da saída |
| ☐ O estado sobrevive ao reinício do contexto, em arquivo, não na janela | ☐ Falha de tool devolve causa nomeada e próxima ação |
| ☐ Subtarefas pesadas rodam em contexto isolado | ☐ Restrições sensíveis vivem na ferramenta, não no prompt |
| ☐ O loop recomeça limpo e lê estado persistido | ☐ Um identificador de rastreamento atravessa runtime, harness e fronteiras |

Como ler: conte um ponto por item presente. Abaixo de cinco, você tem um agente raso com um framework por cima. De cinco a sete, um harness funcional. Oito, um harness pronto para trabalho longo de verdade.

Resumo em uma página

A confusão

Engenharia de harness é frequentemente reduzida a "usar ferramentas", "testar ferramentas" ou "escolher CLI". A redução é comum entre engenheiros de software sólidos sem vivência de projeto de IA em produção, e ela sobrevive porque acerta quase tudo: erra só a parte nova.

A parte nova

No software determinístico, a interface é contrato para o programador. No sistema agêntico, a interface é contexto para o modelo. Descrição de tool vira roteador. Mensagem de erro vira entrada que determina a próxima ação. Formato de retorno vira orçamento de contexto.

Os cinco eixos

System prompt. Contrato de ferramentas. Infraestrutura acoplada. Lógica de orquestração. Hooks e middleware determinísticos. Se você não consegue apontar qual eixo mudou, e o que ficou constante enquanto ele mudava, não foi engenharia de harness.

A evidência

Terminal-Bench 2.0: mesmo modelo, mesmo loop, Top 30 para Top 5 mudando só o harness. Anthropic: 40% de queda no tempo de conclusão reescrevendo a descrição de uma tool. Berkeley: a maioria dos times em produção constrói harness próprio, e confiabilidade é resolvida com design de nível de sistema.

O método

Fixe o modelo por proxy. Selecione tarefas com margem entre 20% e 80%. Varie um eixo. Meça sucesso e turnos. Intercale condições com semente fixa. Reporte intervalo de confiança. Teste o mecanismo, não só o efeito.

A frase que resume

O modelo tem a inteligência. O harness a torna útil. Quando o modelo vira commodity, a variância migra para o que está ao redor dele, e isso é uma superfície de projeto, não uma tarde de leitura de documentação.

Fontes e referências

Vivek Trivedy, *The Anatomy of an Agent Harness*, LangChain, março de 2026: langchain.com/blog/the-anatomy-of-an-agent-harness

LangChain, *Improving Deep Agents with Harness Engineering*, fevereiro de 2026: blog.langchain.com

Terminal-Bench 2.0, leaderboard público: tbench.ai/leaderboard/terminal-bench/2.0

Anthropic Engineering, *How we built our multi-agent research system*, junho de 2025: anthropic.com/engineering/multi-agent-research-system

Melissa Pan et al., *Measuring Agents in Production*, Berkeley, IBM Research e Stanford, arXiv:2512.04123

Birgitta Böckeler, *Harness engineering*: martinfowler.com/articles/harness-engineering.html

Deep Agents, documentação: docs.langchain.com/oss/python/deepagents/overview

Geoffrey Huntley, Ralph Loop, julho de 2025

W. Ross Ashby, *An Introduction to Cybernetics*, Chapman & Hall, 1956

Nota. Claude Code é produto da Anthropic; Deep Agents e LangGraph, da LangChain. Citamos cada um como exemplo do conceito, sem reproduzir material proprietário. Versões e APIs evoluem rápido; a documentação oficial é sempre a referência mais atual.

INCLUSO NA FORMAÇÃO CONTINUADA

Engenharia de Harness para Agentes de IA

O modelo de linguagem é só uma parte do agente. O que separa um chat esperto de um agente que executa trabalho real é o que se constrói ao redor dele. Este curso abre essa caixa.

Como este e-book mostrou, a disciplina vai muito além de apresentar ferramentas. Ela tem eixos de projeto identificáveis, evidência pública mensurável e método experimental próprio. É exatamente o que falta na leitura de quem, até pouco tempo atrás, não acreditava em agentes de IA e hoje escreve sobre o assunto reduzindo tudo ao que já conhecia.

Se você não é o modelo, você é o harness.

Vivek Trivedy · LangChain

8

módulos sequenciais

50/50

teoria e prática

7

harnesses dissecados

1

mini-harness seu, em Python

Começando agora

Os módulos 1 a 4 são teóricos e partem do zero conceitual. Qualquer pessoa tecnicamente curiosa acompanha, sem pré-requisito.

- O que um LLM faz de verdade, e o que ele não consegue sozinho
- A anatomia dos componentes de todo harness de produção
- Engenharia de contexto e a batalha contra o context rot
- Os padrões de loop que sustentam trabalho longo

Já usando as ferramentas

Você já usa Claude Code, Codex ou Cursor, mas quer parar de operar no escuro. Noções de terminal e Python básico ajudam, e os roteiros são guiados passo a passo.

- Claude Code e Codex dissecados por dentro, decisão a decisão
- Deep Agents, OpenCode, Qwen Code e Kimi CLI
- Seu mini-harness do zero, em Python puro
- Orquestração multiagente e o futuro da disciplina

Módulos sequenciais e explicativos. Metade teoria em slides, metade prática com terminal, código e demonstrações ao vivo. Cada módulo encerra com questionário conceitual, e nenhuma questão exige rodar código. Os módulos 1 e 2 já estão disponíveis, e os demais são liberados na sequência.

Disponível como curso avulso ou dentro da Formação Continuada.

harness.scorasacademy.com.br

Quero me inscrever

Scoras Academy. A primeira formação continuada em Engenharia de IA do Brasil.

A Formação reúne dezenas de outros cursos, do fundamento à produção. @scoras_academy

Ver os módulos

- | | |
|---|--|
| 01. O conceito. LLM é commodity, agente é outra coisa. | 05. Claude Code e Codex dissecados por dentro. |
| 02. Anatomia do harness. Os componentes de todo sistema de produção. | 06. O ecossistema aberto: Deep Agents, OpenCode, Qwen Code, Kimi CLI. |
| 03. Engenharia de contexto e context rot. | 07. Seu mini-harness do zero, em Python puro. |
| 04. Padrões de loop e trabalho de horizonte longo. | 08. Orquestração multiagente e o futuro da disciplina. |

Este e-book é material didático da Scoras Academy. As fontes citadas estão na página anterior.